

# ZIN MIN KHANT (Alfred)

## Senior Software Engineer

Enterprise Software · Software Architecture · Identity & Security · AI-Enabled Systems

### PROJECT PORTFOLIO

Bangkok, Thailand | +66 64 041 4110 | zinminkhant.alfred@gmail.com

[LinkedIn](#) | [GitHub](#) | [Portfolio](#)

---

This portfolio describes five production systems I architected and built. Together they cover the two halves of software architecture I most want to develop further at AIT: modernizing large systems that cannot afford downtime, and designing new systems where security, identity, and AI are first-class concerns. The first four are enterprise and platform systems; the fifth is a product I built single-handed under a real platform constraint, included to show the same engineering judgment at a smaller scale. Each project is presented through its business problem, system architecture, technologies, my responsibilities, the hardest engineering challenges, and the results.

**A note on the architecture sections:** they are written as prose rather than drawn diagrams, and they describe real systems running in production. Figures such as user counts, service counts, and page counts are drawn directly from the systems themselves; I have not estimated or rounded them upward.

---

# Cascade ISSARA INSTITUTE · FLAGSHIP MODERNIZATION

*Cross-language re-platforming of a labor-rights analytics product. Node.js, TypeScript, React 19. 2025 to present.*

## BUSINESS PROBLEM

Issara Institute runs a labor-rights analytics platform that protects more than 500,000 workers and covers over 10,000 suppliers across Southeast Asia. The analytics product was built on an aging Python and Django stack that had grown hard to extend and slow to operate, yet it could not be taken offline: partner organizations, auditors, and field teams depend on it daily. The goal was to move the product onto a modern, maintainable stack without a high-risk rewrite and without interrupting a system of record that real worker-protection decisions rely on.

## SYSTEM ARCHITECTURE

I led the re-platforming using a strangler-fig strategy: a new Node.js and TypeScript backend (Express, Sequelize, Zod) was placed alongside the legacy Django system and grew capability by capability, so the old system was retired incrementally rather than replaced in a single cut-over. The backbone of the design is a polyglot, fault-tolerant persistence layer. A primary application database holds the new product's data, while two read-only connections reach into legacy MySQL systems that remain the source of record for existing data. Those legacy connections are wrapped so that they degrade gracefully: when a legacy source is unavailable, new features keep functioning instead of failing, and the degraded state is surfaced rather than hidden. Authentication and identity were re-implemented in Node against the centralized OIDC gateway (described later in this portfolio), with RS256 and JWKS verification, key rotation, and AES-256-GCM response encryption. On the client, the analytics interface is a React 19 single-page application built with Vite and a TanStack data layer, with offline-first synchronization through Dexie and IndexedDB, role-based multi-tenant access, feature flags, internationalization, and geospatial dashboards.

## TECHNOLOGIES

**Backend:** TypeScript, Node.js, Express, Sequelize, Zod; alongside the legacy Django and DRF system. **Data:** a primary application database with read-only connections into two legacy MySQL systems. **Identity:** WorkOS, OIDC, RS256 and JWKS, AES-256-GCM. **Frontend:** React 19, Vite, TypeScript, TanStack, Dexie and IndexedDB, i18n. **Quality and cloud:** Vitest, Playwright, Docker, GitHub Actions.

## MY RESPONSIBILITIES

- Owned the architecture of the modernization: the strangler-fig migration plan, the multi-database persistence design, and the graceful-degradation model.
- Built the new Node and TypeScript backend and re-implemented the authentication and security layer against the central identity gateway.
- Built the React 19 analytics application, including the offline-first synchronization layer and multi-tenant access control.
- Set technical direction on a three-person engineering team, running design reviews and reviewing the other developers' work.

## HARDEST CHALLENGES

- Replacing the stack of a live system of record without downtime. The strangler-fig approach let new and old run side by side and the migration proceed in safe increments.
- Reading from legacy databases that could not be modified and might be unavailable. Read-only connections with timeouts and graceful degradation kept new features working through legacy outages.
- Keeping a half-million-worker dataset usable in the field on poor connectivity. The offline-first client syncs locally and reconciles when the network returns.

## RESULTS

The analytics product now runs on a modern, type-safe stack while the legacy system is retired piece by piece, with no big-bang cut-over and no service interruption for the partners and field teams that depend on it. The fault-tolerant data layer means a legacy outage no longer takes the product down, and the new client works offline in low-connectivity settings. The result is a system that is materially easier to extend, operate, and reason about than the one it replaces.

## Booppa BOOPPA SMART CARE LLC · SOLE TECHNICAL LEAD

*A multi-tenant compliance-evidence SaaS using blockchain and AI. FastAPI, Next.js 14, Polygon. Architected and built end to end.*

### BUSINESS PROBLEM

Compliance evidence has a trust problem. A PDF that asserts "we passed the audit" is easy to forge and carries no independent proof of when it was true. Buyers, auditors, and regulators cannot verify that a vendor's compliance artifact has not been back-dated or altered. Booppa's purpose is to make every piece of evidence independently verifiable and tamper-evident, so a buyer can confirm that a specific document existed in a specific state at a specific time without having to trust Booppa, then wrap that capability in a product that non-technical compliance teams can actually buy and use.

### SYSTEM ARCHITECTURE

As sole technical lead I architected and built the whole platform. The differentiator is a three-layer evidence model. First, a compliance scan (run as an asynchronous task) fetches a target's headers, privacy policy, and cookies, scores them against a set of PDPA dimensions, and uses AI classifiers for risks such as personal-data exposure, with sensitive data redacted before any external model call. Second, the generated evidence PDF is reduced to a deterministic SHA-256 content hash, and every action taken on it is appended to a hash-chained audit log where each row links to the previous row's hash, so altering any past event breaks the chain. Third, the hash is anchored on the Polygon blockchain through a Solidity smart contract I wrote, idempotently so duplicate work does not re-spend gas, and the transaction is surfaced as a QR code that links to a public block explorer for independent verification. Around this core sits a FastAPI backend of 58 routers and more than 70 services, asynchronous Celery and Redis pipelines on two queues with roughly 14 scheduled jobs, a multi-tenant data model, and a provider-agnostic AI router spanning several model vendors with a heuristic fallback that makes no external calls at all. The frontend is a Next.js 14 application of 139 routes with three middleware-enforced authentication zones, HMAC-signed paywall cookies so entitlement state cannot be forged on the client, and real-time notifications over a socket authenticated with a short-lived token. A race-resistant Stripe checkout-to-fulfilment pipeline handles one-time and subscription products across vendor and buyer lanes, including a deferred-intake flow for products that require a buyer brief before fulfilment. The system runs on AWS ECS Fargate behind a Cloudflare Tunnel, with secrets in AWS Secrets Manager and OpenTelemetry and Prometheus observability.

### TECHNOLOGIES

**Backend:** Python, FastAPI, Pydantic, PostgreSQL with SQLAlchemy and Alembic, Redis, Celery. **Blockchain:** web3.py, eth-account, a Solidity EvidenceAnchorV3 contract on Polygon. **AI:** a multi-provider router (Anthropic, OpenAI, and others) with PII redaction before any external call. **Payments and auth:** Stripe, JWT, bcrypt, SAML SSO, HMAC-signed cookies, rate limiting. **Frontend:** Next.js 14, React 18, TypeScript, Tailwind, Recharts, socket.io. **Cloud:** AWS ECS Fargate, Cloudflare Tunnel, Secrets Manager, OpenTelemetry and Prometheus.

### MY RESPONSIBILITIES

- Architected the entire platform: the evidence and anchoring model, the service decomposition, the asynchronous pipelines, the multi-tenant data model, and the deferred billing-to-fulfilment pipeline.
- Built the FastAPI backend, authored the Solidity smart contract and the anchoring service, and built the multi-provider AI router with its redaction boundary.
- Built the Next.js 14 frontend, including the multi-zone authentication, billing, dashboards, and blockchain-verification interface.
- Owned security and cloud: typed JWTs, SAML SSO, HMAC paywall integrity, secrets management, and the ECS and Cloudflare deployment.

### HARDEST CHALLENGES

- Making evidence trustworthy without asking anyone to trust Booppa. The three independent layers (content hash, hash-chained log, public on-chain anchor) make verification independent of the platform.
- Controlling blockchain cost while keeping the proof meaningful. Idempotent anchoring, batch anchoring of many hashes per transaction, and a deliberate force path where a fresh transaction is genuinely required.
- Using AI on sensitive compliance data safely. A redaction boundary strips personal data before any external model call, with a fallback that uses no external calls.
- Handling money correctly. The deferred-intake Stripe flow uses session-scoped state resolution to avoid the classic "paid but nothing fulfilled" and "stuck generating" race conditions.

## RESULTS

Booppa is a complete production SaaS that I architected and built end to end as technical lead: a backend of 58 routers and more than 70 services, a 139-route frontend, a smart contract, a multi-provider AI layer, and a billing pipeline, deployed on managed cloud infrastructure. It produces compliance evidence whose integrity a buyer can verify independently, which is the property the product exists to provide. For me it is the project that demonstrates the widest range, since it spans architecture, blockchain, AI integration, payments, security, and cloud at near-founder ownership.

# Issara Authorizer ISSARA INSTITUTE · IDENTITY KEYSTONE

*A centralized OIDC single-sign-on and identity-federation gateway shared by three products. Flask, WorkOS, AWS Lambda, CDK.*

## BUSINESS PROBLEM

Issara runs several distinct products for the same community of stakeholders: a worker-facing product and two labor-rights monitoring products. Each had, or would have had, its own login, its own user model, and its own session handling. That fragments identity. A user who exists in two products ends up with two disconnected accounts and two logins, there is no single place to revoke a compromised session across products, and each application re-implements OAuth, token verification, and CORS in slightly different and inconsistently secured ways. The need was a single authentication gateway that signs a user in once, reconciles their identity across products, and hands each product a consistent authorization context, without becoming a stateful single point of failure.

## SYSTEM ARCHITECTURE

I designed and built a stateless identity-federation gateway. A login request redirects to WorkOS AuthKit for authentication; the callback exchanges the authorization code for tokens and then enriches the identity by calling each downstream product's user-info endpoint, normalizing and merging the results into a single authorization payload (active flags, groups, user type, and the per-product identifiers). The cross-product correlation is persisted back into the WorkOS user's metadata, which becomes the shared identity ledger, so there is no new central user database to own or keep in sync. That persistence is non-blocking: a metadata write failure never breaks login. Protected requests are verified with RS256 against the WorkOS JWKS endpoint with issuer validation, and the unsigned decode used only to display token expiry is kept deliberately separate from the signature-verified decode that authorizes requests. Because the gateway is the central session authority, it can offer capabilities a per-app login cannot: single-session logout, and a "sign out of all other sessions" operation that enumerates a user's active sessions and revokes every one except the current. The service is packaged as an AWS Lambda function through CDK, and deployed by a GitHub Actions pipeline that assumes roles in separate staging and production AWS accounts via OIDC, so there are no long-lived cloud credentials in CI.

## TECHNOLOGIES

**Language and framework:** Python, Flask. **Identity:** WorkOS AuthKit for OAuth2 and OIDC, PyJWT, RS256 verification via JWKS. **Integration:** server-to-server user-info calls into each product's CMS. **Cloud and CI:** AWS Lambda, AWS CDK, GitHub Actions with multi-account OIDC role assumption. **Patterns:** gateway and backend-for-frontend for auth, identity federation and enrichment, stateless service, graceful degradation, centralized session management.

## MY RESPONSIBILITIES

- Designed the gateway pattern: a stateless federation service that authenticates through WorkOS and enriches per product, with correlation persisted to WorkOS metadata.
- Built the Flask application: the OAuth2 callback orchestration, JWKS-based token verification, downstream user-info aggregation, and central session revocation.
- Owned the security model: RS256 and JWKS with issuer validation, hardened CORS with credentials, and HttpOnly, Secure, SameSite refresh-token cookies.
- Built the serverless packaging and the keyless multi-account deployment pipeline.

## HARDEST CHALLENGES

- Unifying identity across independent products with no shared database. Federate at login and use WorkOS metadata as the shared correlation ledger, so no new central user store has to be owned.
- Staying available when a downstream product is slow or down. Timeouts on every call plus graceful degradation, so a failed fetch yields a null section rather than a failed login.
- Consistent, correct token security across the ecosystem. One JWKS and RS256 verification implementation that every product trusts, instead of three divergent ones.

## RESULTS

The gateway is the architectural keystone that gives Issara's product family one secure single sign-on and one consistent notion of who a user is. It delivers centralized session control, including sign-out-everywhere revocation, which only a central authority can provide, and it does so as a stateless serverless service deployed without any long-lived cloud credentials. It is also the reason authentication looks consistent across the other systems in this portfolio, since they consume the identity layer it provides.

## ILM Platform ISSARA INSTITUTE · ENTERPRISE SYSTEM OF RECORD

*The production labor-rights platform serving 10,000+ suppliers and 500,000+ workers. Django, DRF, React.*

### BUSINESS PROBLEM

The ILM platform is the established, large-scale labor-rights system that monitors ethical supply chains and worker protection across Southeast Asia. It is the production system of record covering more than 10,000 suppliers and more than 500,000 workers, the data backbone that partner organizations and field teams rely on, and the system that Cascade modernizes alongside rather than replaces. Work here had two constant requirements: keep a heavily used enterprise platform reliable and secure, and harden its authentication so a single identity layer could serve the wider product family.

### SYSTEM ARCHITECTURE

The platform is a large Django and Django REST Framework application backed by MySQL, with a substantial domain model spanning suppliers, workers, partners, and monitoring data, and a React frontend. The most security-sensitive work was on authentication. The system runs a dual-provider authentication design that supports both a legacy JWT-with-two-factor scheme and WorkOS-based single sign-on, which allowed a migration toward the centralized identity gateway without a disruptive cut-over. Token verification uses RS256 with JWKS, and the JWKS keys are cached with a time-based expiry and a forced refresh on a verification miss, so key rotation is handled without a network round-trip on every request. I also worked on privilege-escalation hardening, closing paths where a request could assert another user's identity, and on keeping the authorization rules consistent as the platform evolved. The application is deployed across managed cloud infrastructure, with the identity flows aligned to the same WorkOS-based model used by the rest of the ecosystem.

### TECHNOLOGIES

**Backend:** Python, Django, Django REST Framework, MySQL. **Identity and security:** WorkOS AuthKit and JWT with two-factor in a dual-provider design, RS256 and JWKS with key caching and rotation, privilege-escalation hardening. **Frontend:** React. **Cloud:** managed cloud hosting across Google App Engine and AWS.

### MY RESPONSIBILITIES

- Worked on the production labor-rights platform as part of the Tech and Innovation team, focused on backend, identity, and security.
- Hardened the authentication layer: JWKS verification with caching and rotation, and a dual-provider design bridging legacy JWT-2FA and WorkOS SSO.
- Closed privilege-escalation paths and kept authorization rules consistent as the platform changed.
- Aligned the platform's identity flows with the centralized gateway so it became one of three products sharing a single sign-on.

### HARDEST CHALLENGES

- Evolving authentication on a live, heavily used platform without locking out a large, distributed user base. The dual-provider design allowed the legacy and new schemes to run together during migration.
- Rotating signing keys without paying a network cost on every request. JWKS caching with expiry and forced refresh on a miss keeps verification fast and correct.
- Hardening a large existing codebase against privilege escalation while keeping legitimate access working.

## RESULTS

The platform continues to serve as the reliable system of record for a half-million-worker dataset while its authentication has been hardened and aligned to a single shared identity model. That alignment is what made the strangler-fig modernization in Cascade possible: the legacy platform and the new stack can share one login and one notion of identity, so the system can be modernized incrementally rather than replaced in a destabilizing rewrite.

# SingPost "MyPostman" WUNDERFAUKS (SINGAPORE) · SOLE DEVELOPER

*A resident-feedback platform for Singapore Post built as a relational application on WordPress. PHP, custom InnoDB schema, REST. 2023 to 2024.*

## BUSINESS PROBLEM

Singapore Post wanted residents to give structured feedback on the specific person who delivers their mail, not a generic contact form. That is harder than it sounds: a resident identifies by address (postal code), but feedback has to attach to a postman (a staff member), and the link between the two runs through delivery beats, the route units that each cover a set of postal codes and are assigned to one postman. The public must never browse or enumerate staff; a resident should only ever see the single postman who serves the postal code they entered. There was also a modernization constraint: the previous platform had been built by an outside partner who stopped supporting it and never handed over full access, so the brief was to rebuild the capability on a system the client owns and can run themselves.

## SYSTEM ARCHITECTURE

I treated this as a relational application that happens to live inside WordPress, not as a website. The core is a bespoke schema of four InnoDB tables created on theme initialization: a postman table (the staff record), a beat table (the route unit), a postal-code table that maps each address to a beat, and a postman-to-beat assignment join with foreign keys to both. That join is what lets one postal code resolve to one accountable person. The public resolution flow is a three-step server-side lookup: postal code to beat, beat to assigned staff member, staff member to profile, each step a prepared statement so input from the open internet never reaches SQL as a string, and the resident only ever sees the single resolved postman. Two custom REST endpoints under a private namespace accept writes, one for per-postman ratings and one for general feedback with constrained file uploads; both verify a Google reCAPTCHA token server-side before writing and sanitize every field. The admin side uses custom post types and meta boxes that mirror their values into the relational tables, with nonce and capability checks on every write, a CSV bulk import so the client can load thousands of postman, beat, and postal-code rows without hand entry, and read-only review screens so submitted feedback cannot be quietly altered after the fact.

## TECHNOLOGIES

**Platform and language:** PHP, WordPress with a hand-written custom theme (no page builder). **Data:** four custom InnoDB tables with foreign keys and indexes, prepared statements throughout (PDO for the public lookup, the WordPress database layer for admin CRUD). **API:** custom WordPress REST endpoints for ratings and feedback. **Front end:** jQuery and AJAX form submission, Google reCAPTCHA v2. **Admin:** custom post types and meta boxes, capability-checked CRUD, CSV bulk import.

## MY RESPONSIBILITIES

- Sole developer. Designed the four-table relational schema and built the theme, the public resolution flow, and the admin and CMS end to end.
- Built the address-to-postman resolution as multi-step prepared-statement lookups, exposing exactly one postman per postal code and never a browsable staff list.
- Built the two public write endpoints with server-side reCAPTCHA verification, input sanitization, and constrained file uploads.
- Built the admin maintenance tooling: capability-checked CRUD, CSV bulk import for thousands of records, and tamper-resistant read-only feedback views.

## HARDEST CHALLENGES

- Letting a resident rate a person they cannot name. Modeling the operational reality (postal code to beat to postman) as a relational schema and resolving it server-side meant the resident only had to supply an address.
- Rebuilding a live capability with no access to the system it replaced. I reconstructed the operational model from first principles as an owned schema, then gave the client CSV import so they could load their own data rather than depend on a platform they could not reach.
- Accepting writes from the open internet without opening the door to spam or injection. Server-side reCAPTCHA on both endpoints, sanitization on every field, prepared statements on every query, and file-upload limits.

## RESULTS

The platform gives Singapore Post a resident-feedback system they own and can maintain themselves, built around a real relational domain model rather than a contact form. It resolves any Singapore postal code to the single accountable postman through a beat-assignment join, carries thousands of operational records across postmen, beats, and postal-code mappings, and accepts public submissions through hardened, bot-protected write endpoints. I include it as the smallest of the five systems to show that the same instincts, model the domain honestly, expose the least surface necessary, and protect every write path, hold even when the platform is a CMS rather than a stack of my own choosing.